TASK 6 – Read and Print Data Correctly 4 Marks

Bloom's Level: K3 – Apply Design the input and output of the program using scanf and printf.

Discuss:

• The correct format specifier for each data type (%d, %u, %ld, %lld, %f, %lf, %c, %s)

• What happens when a wrong specifier is used (for example %d for a long), shown by a test

• How to check the return value of scanf to detect invalid input such as letters

• How to print rupees and paise from an integer amount in paise (for example Rs 162.50) Alternative Concept and Complexity Analysis If each input line were read with fgets() and then converted with sscanf() or strtol() instead of reading directly with scanf(), what would the solution be? Analyse the time to process a line of length L and the buffer space needed, and compare how each approach recovers from invalid input. Evidence Expected Correct format specifiers, input validation using the return value of scanf, and a correctly formatted bill

Answer :

1. Correct Format Specifiers

Format specifiers tell scanf() and printf() what type of data is being read or displayed.

Data type	**Format specifier**	**Example**
int	%d	scanf("%d", &age);
unsigned int	%u	printf("%u", units);
long	%ld	printf("%ld", amount);
long long	%lld	printf("%lld", number);
float	%f	scanf("%f", &value);
double	%lf in scanf(); %f in printf()	scanf("%lf", &value);
char	%c	scanf(" %c", &choice);
String	%s	scanf("%s", name);

The correct format specifier prevents incorrect input handling and output.

2. Problems with Incorrect Format Specifiers

Using the wrong format specifier can cause incorrect results or undefined behaviour. For example, using %d to print a long value is incorrect; %ld must be used.

3. Input Validation Using scanf()

The return value of scanf() indicates how many input items were successfully read.

Example:

```
int units;


if (scanf("%d", &units) != 1) {
   printf("Invalid input. Enter an integer.\n");
   return 1;
}
```

If the user enters letters instead of an integer, scanf() returns 0. Checking this value prevents the program from processing invalid input.

4. Printing Rupees and Paise

Money can be stored as an integer number of paise to avoid floating-point rounding errors.

For example, 16250 paise represents ₹162.50.

```
long paise = 16250;


printf("Rs %ld.%02ld\n", paise / 100, paise % 100);
```

Output:

```
Rs 162.50
```

Integer division obtains the rupees, and the remainder gives the paise.

5. Alternative Concept: fgets() and strtol()

Instead of reading input directly using scanf(), the program can read a complete line using fgets() and convert it using strtol() or sscanf().

- fgets() reads the input line into a character buffer.

- strtol() converts the text into an integer and helps detect invalid characters and range errors.
- sscanf() can extract values from the stored line, but its return value must also be checked.

This approach makes it easier to reject invalid input and clear the entire line before requesting another entry. With direct scanf(), invalid characters may remain in the input stream and cause repeated failures if not handled correctly.

6. Time and Space Complexity

Let (L) be the number of characters in an input line.

Method	Time complexity	Extra space	Invalid-input handling
scanf()	Depends on the input length and conversion	Usually small	Requires checking the return value and handling leftover input
fgets() + strtol()	(O(L))	(O(L)) for the input buffer	Easier to validate the entire line and reject trailing characters

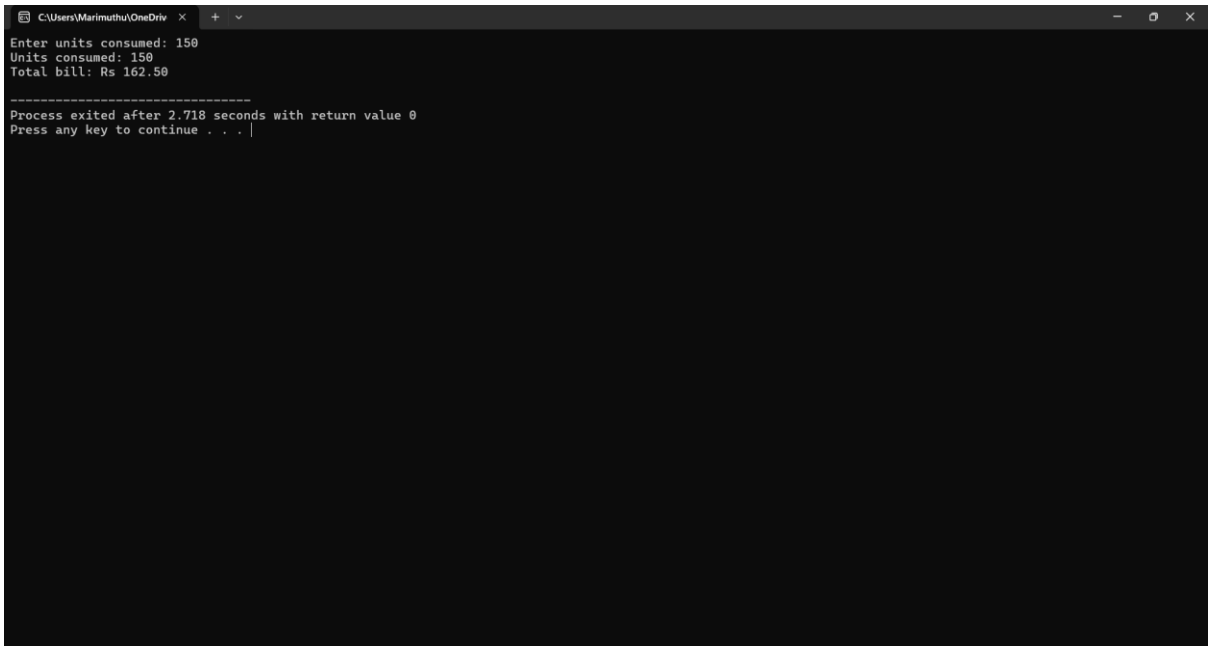
The input buffer must be large enough for the expected line, including the terminating null character. If the line exceeds the buffer, the remaining characters must be handled separately.

Code :

```
#include <stdio.h>
int main(void)
{
  int units;
  long paise;
  printf("Enter units consumed: ");
  if (scanf("%d", &units) != 1) {
    printf("Invalid input. Enter an integer.\n");
    return 1;
  }
```

```
    if (units < 0) {
        printf("Units cannot be negative.\n");
        return 1;
    }
    paise = 5000L;
    if (units > 200)
        paise += (long)(units - 200) * 450;
    if (units > 100) {
        int slab2 = units < 200 ? units - 100 : 100;
        paise += (long)slab2 * 225;
    }
    printf("Units consumed: %d\n", units);
    printf("Total bill: Rs %ld.%02ld\n",
           paise / 100, paise % 100);
    return 0;
}
```

Output:



Conclusion

Correct format specifiers, validation of the scanf() return value, and integer-based money calculations improve the reliability of a C program. Using fgets() with strtol() provides more control over invalid input.